

MODULE-2

RELATIONAL DATABASES AND SQL

Relational Algebra: Select, Project, Join, Union, Intersection, Difference, Rename, Division.

SQL Basics: DDL: Create, Drop, Alter, Truncate; DML: Insert, Delete, Update; SQL Clauses: Where, Order By, Group By; Constraints: Not Null, Unique, Primary, Foreign Key; Aggregate functions: SUM, AVG, COUNT, MIN, MAX.

Relational Algebra Operations

Relational algebra is a **formal query language** for relational databases. It consists of **operations that take one or more relations (tables) as input and produce a relation as output.**

1. SELECT (σ) – Selection

- **Purpose:** Choose **rows that satisfy a condition.**
- **Symbol:** σ
- **Syntax:**

$$\sigma_{\text{condition}}(\text{Relation})$$

Example:

Table: SNACKS

```
mysql> -- Create SNACKS table
mysql> CREATE TABLE SNACKS (
  -> SNACK_ID INT NOT NULL PRIMARY KEY,          -- Unique identifier for each snack
  -> SNACK_NAME VARCHAR(50) NOT NULL,            -- Name of the snack
  -> COMPANY VARCHAR(50) NOT NULL,               -- Name of the company
  -> COST DECIMAL(5,2) NOT NULL CHECK (COST > 0) -- Cost of snack, must be positive
  -> );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COMPANY, COST) VALUES
  -> (1, 'Chips', 'Lays', 20),
  -> (2, 'Cookies', 'Oreo', 30),
  -> (3, 'Chocolate', 'Cadbury', 50);
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> -- Snacks costing more than 25
mysql> SELECT * FROM SNACKS WHERE COST > 25;
+-----+-----+-----+-----+
| SNACK_ID | SNACK_NAME | COMPANY | COST |
+-----+-----+-----+-----+
| 2 | Cookies | Oreo | 30.00 |
| 3 | Chocolate | Cadbury | 50.00 |
+-----+-----+-----+-----+
```

- Select snacks with COST > 25:

$$\sigma_{\text{COST}>25}(\text{SNACKS})$$

```
mysql> -- Snacks costing more than 25
mysql> SELECT * FROM SNACKS WHERE COST > 25;
+-----+-----+-----+-----+
| SNACK_ID | SNACK_NAME | COMPANY | COST |
+-----+-----+-----+-----+
| 2 | Cookies | Oreo | 30.00 |
| 3 | Chocolate | Cadbury | 50.00 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

2. PROJECT (π) – Projection

- **Purpose:** Choose **specific columns.**
- **Symbol:** π
- **Syntax:**

$$\pi_{\text{column1, column2, ...}}(\text{Relation})$$

Example:

- Project SNACK_NAME and COST:

$$\pi_{SNACK_NAME, COST}(SNACKS)$$

```
mysql> SELECT SNACK_NAME, COST
-> FROM SNACKS;
+-----+-----+
| SNACK_NAME | COST |
+-----+-----+
| Chips      | 20.00 |
| Cookies    | 30.00 |
| Chocolate  | 50.00 |
+-----+-----+
3 rows in set (0.00 sec)
```

3. JOIN (⋈) – Combining Tables

- Purpose:** Combine rows from **two relations based on a condition**.
- Syntax:**

$$R \bowtie_{R.key=S.key} S$$

Example:

- Tables: SNACKS and COMPANIES

COMPANY_ID	COMPANY_NAME
101	Lays
102	Oreo

- Join on COMPANY_ID:

$$SNACKS \bowtie_{SNACKS.COMPANY_ID=COMPANIES.COMPANY_ID} COMPANIES$$

Output:

```
+-----+-----+-----+
| SNACK_NAME | COMPANY_NAME | COST |
+-----+-----+-----+
| Chips      | Lays         | 20.00 |
| Cookies    | Oreo         | 30.00 |
| Chocolate  | Cadbury      | 50.00 |
+-----+-----+-----+
```

```
mysql> CREATE TABLE COMPANIES (
->     COMPANY_ID INT PRIMARY KEY,
->     COMPANY_NAME VARCHAR(50) NOT NULL,
->     COUNTRY VARCHAR(50)
-> );
Query OK, 0 rows affected (0.16 sec)

mysql> INSERT INTO COMPANIES VALUES
-> (101, 'Lays', 'USA'),
-> (102, 'Oreo', 'USA'),
-> (103, 'Cadbury', 'UK'),
-> (104, 'Parle', 'India');
Query OK, 4 rows affected (0.07 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE TABLE SNACKS (
  ->     SNACK_ID INT PRIMARY KEY,
  ->     SNACK_NAME VARCHAR(50) NOT NULL,
  ->     COMPANY_ID INT,
  ->     COST DECIMAL(5,2) NOT NULL,
  ->     FOREIGN KEY (COMPANY_ID) REFERENCES COMPANIES(COMPANY_ID)
  -> );
Query OK, 0 rows affected (0.15 sec)

mysql> INSERT INTO SNACKS VALUES
  -> (1, 'Chips', 101, 20),
  -> (2, 'Cookies', 102, 30),
  -> (3, 'Chocolate', 103, 50);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

1. INNER JOIN

- Returns rows with matching keys in both tables.

```
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
  -> FROM SNACKS S
  -> INNER JOIN COMPANIES C
  -> ON S.COMPANY_ID = C.COMPANY_ID;
+-----+-----+-----+
| SNACK_NAME | COMPANY_NAME | COST |
+-----+-----+-----+
| Chips      | Lays         | 20.00 |
| Cookies    | Oreo         | 30.00 |
| Chocolate  | Cadbury      | 50.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

2. THETA JOIN

- A general join with a condition other than equality.

```
mysql> -- Snacks costing more than 25 and match any company (example)
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
  -> FROM SNACKS S
  -> JOIN COMPANIES C
  -> ON S.COMPANY_ID = C.COMPANY_ID AND S.COST > 25;
+-----+-----+-----+
| SNACK_NAME | COMPANY_NAME | COST |
+-----+-----+-----+
| Cookies    | Oreo         | 30.00 |
| Chocolate  | Cadbury      | 50.00 |
+-----+-----+-----+
2 rows in set (0.07 sec)
```

3. EQUI JOIN

- A special case of theta join with only equality conditions.
- MySQL INNER JOIN with = is equi join.

```
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
  -> FROM SNACKS S
  -> JOIN COMPANIES C
  -> ON S.COMPANY_ID = C.COMPANY_ID;
+-----+-----+-----+
| SNACK_NAME | COMPANY_NAME | COST |
+-----+-----+-----+
| Chips      | Lays         | 20.00 |
| Cookies    | Oreo         | 30.00 |
| Chocolate  | Cadbury      | 50.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

4. NATURAL JOIN

- Automatically joins tables on columns with the same name.

```
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
-> FROM SNACKS S
-> NATURAL JOIN COMPANIES C;
```

SNACK_NAME	COMPANY_NAME	COST
Chips	Lays	20.00
Cookies	Oreo	30.00
Chocolate	Cadbury	50.00

3 rows in set (0.00 sec)

5. LEFT OUTER JOIN

- Returns all rows from left table (SNACKS) and matching rows from right table (COMPANIES).
- Non-matches → NULL.

```
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
-> FROM SNACKS S
-> LEFT JOIN COMPANIES C
-> ON S.COMPANY_ID = C.COMPANY_ID;
```

SNACK_NAME	COMPANY_NAME	COST
Chips	Lays	20.00
Cookies	Oreo	30.00
Chocolate	Cadbury	50.00

3 rows in set (0.00 sec)

6. RIGHT OUTER JOIN

- Returns all rows from right table (COMPANIES) and matching rows from left table (SNACKS).

```
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
-> FROM SNACKS S
-> RIGHT JOIN COMPANIES C
-> ON S.COMPANY_ID = C.COMPANY_ID;
```

SNACK_NAME	COMPANY_NAME	COST
Chips	Lays	20.00
Cookies	Oreo	30.00
Chocolate	Cadbury	50.00
NULL	Parle	NULL

4 rows in set (0.00 sec)

7. FULL OUTER JOIN (Simulated in MySQL)

```
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
-> FROM SNACKS S
-> LEFT JOIN COMPANIES C
-> ON S.COMPANY_ID = C.COMPANY_ID
->
-> UNION
->
-> SELECT S.SNACK_NAME, C.COMPANY_NAME, S.COST
-> FROM SNACKS S
-> RIGHT JOIN COMPANIES C
-> ON S.COMPANY_ID = C.COMPANY_ID;
+-----+-----+-----+
| SNACK_NAME | COMPANY_NAME | COST |
+-----+-----+-----+
| Chips      | Lays         | 20.00 |
| Cookies    | Oreo         | 30.00 |
| Chocolate  | Cadbury      | 50.00 |
| NULL       | Parle        | NULL  |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

8. SELF JOIN

- Join a table with itself.
- Useful to compare rows within the same table.

```
mysql> -- Example: Compare snacks costing more than another snack
mysql> SELECT S1.SNACK_NAME AS Snack1, S2.SNACK_NAME AS Snack2
-> FROM SNACKS S1
-> JOIN SNACKS S2
-> ON S1.COST > S2.COST;
+-----+-----+
| Snack1 | Snack2 |
+-----+-----+
| Chocolate | Chips |
| Cookies   | Chips |
| Chocolate | Cookies |
+-----+-----+
3 rows in set (0.00 sec)
```

9. CROSS JOIN

- Returns all possible combinations of rows from both tables (Cartesian product).

```
mysql> SELECT S.SNACK_NAME, C.COMPANY_NAME
-> FROM SNACKS S
-> CROSS JOIN COMPANIES C;
+-----+-----+
| SNACK_NAME | COMPANY_NAME |
+-----+-----+
| Chocolate  | Lays         |
| Cookies    | Lays         |
| Chips      | Lays         |
| Chocolate  | Oreo         |
| Cookies    | Oreo         |
| Chips      | Oreo         |
| Chocolate  | Cadbury      |
| Cookies    | Cadbury      |
| Chips      | Cadbury      |
| Chocolate  | Parle        |
| Cookies    | Parle        |
| Chips      | Parle        |
+-----+-----+
12 rows in set (0.00 sec)
```

Number of rows = rows in SNACKS × rows in COMPANIES = 3 × 4 = 12

4. UNION (U)

- **Purpose:** Combine rows from **two relations** (duplicates removed).
- **Syntax:**

$$R \cup S$$

```
mysql> select * from snacks_available;
+-----+-----+-----+
| SNACK_NAME | COMPANY_ID | COST |
+-----+-----+-----+
| Chips      | 101        | 20.00 |
| Cookies    | 102        | 30.00 |
| Chocolate  | 103        | 50.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> select * from snacks_out_of_stock;
+-----+-----+
| SNACK_NAME | COMPANY_ID |
+-----+-----+
| Chips      | 101        |
| Biscuit    | 104        |
| KitKat     | 105        |
+-----+-----+
3 rows in set (0.00 sec)
```

Example:

```
mysql> SELECT SNACK_NAME FROM SNACKS_AVAILABLE
-> UNION
-> SELECT SNACK_NAME FROM SNACKS_OUT_OF_STOCK;
+-----+
| SNACK_NAME |
+-----+
| Chips      |
| Cookies    |
| Chocolate  |
| Biscuit    |
| KitKat     |
+-----+
5 rows in set (0.00 sec)

mysql> SELECT SNACK_NAME, COMPANY_ID FROM SNACKS_AVAILABLE
-> UNION
-> SELECT SNACK_NAME, COMPANY_ID FROM SNACKS_OUT_OF_STOCK;
+-----+-----+
| SNACK_NAME | COMPANY_ID |
+-----+-----+
| Chips      | 101        |
| Cookies    | 102        |
| Chocolate  | 103        |
| Biscuit    | 104        |
| KitKat     | 105        |
+-----+-----+
5 rows in set (0.00 sec)
```

5. INTERSECTION (∩)

- **Purpose:** Return **rows common to both relations**.
- **Syntax:**

$$R \cap S$$

Example: Both are out of stock

```
mysql> SELECT A.SNACK_NAME
-> FROM SNACKS_AVAILABLE A
-> INNER JOIN SNACKS_OUT_OF_STOCK O
-> ON A.SNACK_NAME = O.SNACK_NAME;
+-----+
| SNACK_NAME |
+-----+
| Chips      |
+-----+
1 row in set (0.00 sec)
```

- Intersection = rows in both tables (subset of JOIN sometimes).
- JOIN = combine columns based on condition (wider table).

6. DIFFERENCE (-)

- **Purpose:** Return rows in the first relation but not in the second.
- **Syntax:**

$$R - S$$

Example:

Snacks available but not out-of-stock:

```
mysql> SELECT SNACK_NAME
-> FROM SNACKS_AVAILABLE
-> WHERE SNACK_NAME NOT IN (SELECT SNACK_NAME FROM SNACKS_OUT_OF_STOCK);
+-----+
| SNACK_NAME |
+-----+
| Cookies    |
| Chocolate  |
+-----+
2 rows in set (0.00 sec)
```

SNACK_NAME	COMPANY
Cookies	Oreo
Chocolate	Cadbury

7. RENAME (ρ)

- **Purpose:** Assign a new name to a relation or attribute.
- **Symbol:** ρ
- **Syntax:**

Example:

$$\rho_{ALL_SNACKS}(SNACKS)$$

- Renames relation SNACKS as ALL_SNACKS.

```
mysql> SELECT * FROM SNACKS_OUT_OF_STOCK;
+-----+-----+
| SNACK_NAME | COMPANY_ID |
+-----+-----+
| Chips      | 101        |
| Biscuit    | 104        |
| KitKat     | 105        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> select snack_name as name, company_id as cid from snacks_out_of_stock;
+-----+-----+
| name      | cid      |
+-----+-----+
| Chips     | 101      |
| Biscuit   | 104      |
| KitKat    | 105      |
+-----+-----+
3 rows in set (0.00 sec)
```

8. DIVISION (÷)

- **Purpose:** Find tuples in **one relation that match all tuples in another relation**.
- **Syntax:**

$$R \div S$$

Example:

- $\text{SNACKS_SOLD}(\text{SNACK_NAME}, \text{STORE}) \div \text{STORES}(\text{STORE}) \rightarrow \text{Snacks sold in all stores.}$

```
mysql> SELECT * FROM SNACKS_AVAILABLE;
+-----+-----+-----+
| SNACK_NAME | COMPANY_ID | COST |
+-----+-----+-----+
| Chips      | 101        | 20.00 |
| Cookies    | 102        | 30.00 |
| Chocolate  | 103        | 50.00 |
| Chips      | 104        | 25.00 |
| Chips      | 105        | 30.00 |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT * FROM SNACKS_OUT_OF_STOCK;
+-----+-----+
| SNACK_NAME | COMPANY_ID |
+-----+-----+
| Chips      | 101        |
| Biscuit    | 104        |
| KitKat     | 105        |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT SNACK_NAME
-> FROM SNACKS_AVAILABLE
-> WHERE COMPANY_ID IN (SELECT COMPANY_ID FROM SNACKS_OUT_OF_STOCK)
-> GROUP BY SNACK_NAME
-> HAVING COUNT(DISTINCT COMPANY_ID) = (SELECT COUNT(DISTINCT COMPANY_ID) FROM SNACKS_OUT_OF_STOCK);
+-----+
| SNACK_NAME |
+-----+
| Chips      |
+-----+
1 row in set (0.00 sec)
```

Only **Chips** appears because it is sold in all stores listed in STORES.

Summary – Relational Algebra Operations

Operation	Symbol	Purpose	Snack Example Use Case
Select	σ	Choose rows satisfying a condition	Snacks with COST > 25
Project	π	Choose specific columns	Snack names and cost
Join	$\bowtie$	Combine tables on matching columns	SNACKS $\bowtie$ COMPANIES
Union	$\cup$	Combine rows from two tables, remove duplicates	Available $\cup$ Out-of-stock snacks
Intersection	$\cap$	Rows common in both tables	Snacks both available & out-of-stock
Difference	$-$	Rows in first table but not in second	Snacks available but not out-of-stock
Rename	ρ	Rename relation or columns	$\rho_ALL_SNACKS(SNACKS)$
Division	$\div$	Rows in first table matching all rows in second	Snacks sold in all stores

SQL Basics

Structured Query Language (SQL) is a **standard programming language** used for **storing, managing and manipulating data** in a **Relational Database Management System (RDBMS)**.

It enables users to perform essential operations such as **creating, reading, updating and deleting data (CRUD operations)**, which are fundamental for handling relational databases efficiently.

SQL is widely used in database systems such as **MySQL, Oracle, Informix, Microsoft Access and SQL Server**. It is considered a **universal language for relational databases** due to its **simple, English-like syntax**, allowing users to **query and interact with data easily and effectively**.

What is SQL?

Structured Query Language (SQL) follows certain **rules and characteristics** that make it effective for managing relational databases:

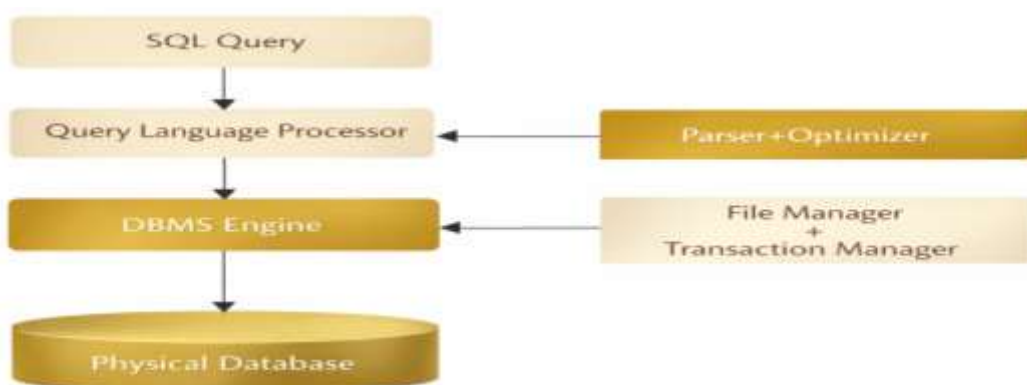
- **SQL is not case sensitive**, meaning keywords can be written in **uppercase or lowercase**. However, it is a common practice to write **SQL keywords in uppercase** for better readability.
- **SQL statements are flexible with text lines**, meaning a single SQL statement can be written **on one line or spread across multiple lines**.
- Using SQL statements, users can perform **most database operations**, such as **data insertion, retrieval, updating and deletion**.

- SQL is based on the principles of **tuple relational calculus** and **relational algebra**, which form the **theoretical foundation of relational databases**.

What is SQL Process?

- The **SQL process** describes how an **SQL command is executed** within a **Relational Database Management System (RDBMS)**. When a query is submitted, the system determines the **most efficient way to execute the request** and the **SQL engine interprets and processes the command** accordingly.
- During this process, several important components are involved, including the **Query Engine**, **Optimization Engine**, **Query Dispatcher** and **Classic Engine**. These components work together to **analyze, optimize and execute SQL queries efficiently**.
- The **SQL Query Engine** is responsible for handling **SQL-based queries**, while the **Classic Query Engine** manages **non-SQL queries**. However, the SQL Query Engine does **not handle logical file operations**, which are managed separately within the system.

SQL Process



What are Advantages of SQL?

- High speed
- No coding needed
- Well defined standards
- Portability
- Interactive language
- Multiple data view

What is SQL Datatype?

An **SQL Datatype** is used to **define the type of values** that a **column can store** in a database table. It ensures that only **valid and appropriate data** is entered into each column.

Every column in a table must have a **unique name** and an associated **datatype**, which specifies whether the data will be **numeric, character, date or other formats**. This helps in maintaining **data integrity and consistency** within the database.



SQL Commands

SQL commands are **instructions used to communicate with a database**. They enable users to **perform specific tasks, functions and data queries** efficiently within a Relational Database Management System (RDBMS).

Using SQL commands, various operations can be performed such as **creating tables, inserting data into tables, modifying existing tables, deleting tables and retrieving data**. Additionally, SQL allows users to **set permissions and control access**, ensuring proper **security and management of the database**.

Types of SQL Commands

- There are five types of SQL commands: DDL, DML, DCL, TCL and DQL.



Data Definition Language (DDL)

Data Definition Language (DDL) is a category of **SQL commands used to define and modify the structure of database objects**, such as tables. It includes operations like **creating a table, altering its structure and deleting a table**.

All DDL commands are **auto-committed**, which means that once a command is executed, the changes are **permanently saved in the database** and cannot be rolled back.

The commonly used DDL commands include **CREATE, ALTER, DROP and TRUNCATE**, which are used to **manage and control the structure of database tables**.

CREATE

The **CREATE** command is used to **create a new table** in the database.

Syntax:

CREATE TABLE table_name (column_name datatype, column_name datatype, ...);

Example:

```
mysql> CREATE TABLE SNACK (  
->     Quantity INT,  
->     Snack_Name VARCHAR(100),  
->     Price DECIMAL(10,2),  
->     Expiry_Date DATE  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> DESC SNACK;  
+-----+-----+-----+-----+-----+-----+  
| Field          | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| Quantity       | int           | YES  |     | NULL    |       |  
| Snack_Name     | varchar(100)  | YES  |     | NULL    |       |  
| Price          | decimal(10,2) | YES  |     | NULL    |       |  
| Expiry_Date    | date          | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.03 sec)
```

DROP

The **DROP** command is used to **delete an entire table**, including both its **structure and all records stored in it**.

Syntax:

DROP TABLE table_name;

Example (Snack Table:

```
mysql> drop table snack;  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> DESC SNACK;  
ERROR 1146 (42S02): Table 'ss.snack' doesn't exist  
mysql> |
```

ALTER

The **ALTER** command is used to **modify the structure of an existing table**. It can be used to **add a new column or modify an existing column's datatype or size**.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

ALTER TABLE table_name MODIFY (column_name datatype);

Example:

```
mysql> DESC SNACK;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Quantity   | int           | YES  |     | NULL    |       |
| Snack_Name | varchar(100)   | YES  |     | NULL    |       |
| Price      | decimal(10,2) | YES  |     | NULL    |       |
| Expiry_Date | date          | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> ALTER TABLE SNACK
-> MODIFY Snack_Name VARCHAR(150);
Query OK, 0 rows affected (0.01 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> DESC SNACK;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Quantity   | int           | YES  |     | NULL    |       |
| Snack_Name | varchar(150)   | YES  |     | NULL    |       |
| Price      | decimal(10,2) | YES  |     | NULL    |       |
| Expiry_Date | date          | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

TRUNCATE

The **TRUNCATE** command is used to **remove all rows from a table** and **free the space occupied by the table data**. It deletes data **quickly and efficiently**, but the **table structure remains intact**.

Syntax:

TRUNCATE TABLE table_name;

Example:

```
mysql> TRUNCATE TABLE SNACKS_AVAILABLE;
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT * FROM SNACKS_AVAILABLE;
Empty set (0.01 sec)
```

Data Manipulation Language (DML)

Data Manipulation Language (DML) consists of **SQL commands used to modify and manage data** stored in a database. It is responsible for **all types of changes performed on the data**, such as adding, updating or removing records.

Unlike DDL commands, DML commands are **not auto-committed**, which means the changes made can be **controlled using transactions** and can be **rolled back if needed**.

The commonly used DML commands include **INSERT, UPDATE and DELETE**, which are used to **insert new data, modify existing data and remove data from tables**.

INSERT

The **INSERT** statement is used to **add new data (rows) into a table**. It allows users to **insert values into specific columns or all columns** of a table.

Syntax:

INSERT INTO table_name (column1, column2, column3, ...) VALUES (value1, value2, value3, ...);

or

INSERT INTO table_name VALUES (value1, value2, value3, ...);

Example:

```
mysql> INSERT INTO SNACK (Snack_ID, Snack_Name, Price, Quantity, Expiry_Date)
-> VALUES (103, 'Cookies', 25, 60, '2026-10-20');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO SNACK (Snack_ID, Snack_Name, Price, Quantity, Expiry_Date)
-> VALUES (104, 'Chocolate', 50, 30, '2027-01-15');
Query OK, 1 row affected (0.01 sec)

mysql> select * from snack;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 101 | Chips | NULL | 20.00 | 2026-12-31 |
| 102 | Biscuits | NULL | 30.00 | 2026-11-15 |
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

UPDATE

The **UPDATE** command is used to **modify or change existing values** in a table. It allows users to **update one or more columns** based on a specific condition.

Syntax:

UPDATE table_name SET column_name1 = value1, column_name2 = value2, ...

WHERE condition;

Example (Snack Table):

UPDATE SNACK SET Price = 25 WHERE Snack_ID = 101;

```
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 101 | Chips | NULL | 25.00 | 2026-12-31 |
| 102 | Biscuits | NULL | 30.00 | 2026-11-15 |
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
+-----+-----+-----+-----+-----+
```

DELETE

DELETE is a DML (Data Manipulation Language) command used to **remove one or more existing rows from a table based on a specified condition**. It does not delete the table structure only the data inside it.

Syntax:

DELETE FROM table_name WHERE condition;

1. Delete a specific row

DELETE FROM SNACK WHERE Snack_ID = 101;

```
mysql> DELETE FROM SNACK WHERE Snack_ID = 101;
Query OK, 1 row affected (0.01 sec)

mysql> select * from snack;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 102 | Biscuits | NULL | 30.00 | 2026-11-15 |
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

2. Delete based on condition

DELETE FROM SNACK

WHERE Price > 30;

```
mysql> DELETE FROM SNACK WHERE Price > 30;
Query OK, 1 row affected (0.01 sec)

mysql> select * from snack;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 102 | Biscuits | NULL | 30.00 | 2026-11-15 |
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

3. Delete expired snacks

DELETE FROM SNACK WHERE Expiry_Date < '2026-11-01';

```
mysql> DELETE FROM SNACK WHERE Expiry_Date < '2026-11-01';
Query OK, 1 row affected (0.01 sec)

mysql> select * from snack;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 102 | Biscuits | NULL | 30.00 | 2026-11-15 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

4. Delete NULL quantity rows

DELETE FROM SNACK WHERE Quantity IS NULL;

```
mysql> DELETE FROM SNACK WHERE Quantity IS NULL;
Query OK, 1 row affected (0.01 sec)

mysql> select * from snack;
Empty set (0.00 sec)

mysql> DESC snack;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Snack_ID | int | NO | PRI | NULL | |
| Snack_Name | varchar(50) | NO | | NULL | |
| Quantity | int | YES | | NULL | |
| Price | decimal(10,2) | NO | | NULL | |
| Expiry_Date | date | NO | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.03 sec)
```

5. Delete ALL rows (no WHERE)

DELETE FROM SNACK;

Key Differences to Remember

Feature	DELETE	DROP	TRUNCATE
Removes rows	☑ Yes	☑ Yes (all)	☑ Yes (all)
Removes structure	✗ No	☑ Yes	✗ No
WHERE clause	☑ Yes	✗ No	✗ No
Can rollback	☑ Yes	✗ No	✗ No
DML / DDL	DML	DDL	DDL

SQL Clauses



SQL GROUP BY Statement

The **GROUP BY** statement in SQL is used to **arrange identical data into groups**. It is commonly used **with aggregation functions** like COUNT(), SUM(), AVG(), MAX(), and MIN() to **summarize data**.

- **GROUP BY** is used with the **SELECT statement**.
- It **follows the WHERE clause** (if any) and **precedes the ORDER BY clause**.

Syntax:

```
SELECT column1, AGGREGATE_FUNCTION(column2) FROM table_name WHERE  
condition GROUP BY column1 ORDER BY column1;
```

Example:

Suppose we have a SNACK table with columns Snack_Name and Price. We want to **count the number of snacks for each price**.

```
SELECT Price, COUNT(*) FROM SNACK GROUP BY Price ORDER BY Price;
```

```
mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
| 105 | Biscuits | 40 | 35.00 | 2026-11-15 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT Price, COUNT(*) FROM SNACK GROUP BY Price ORDER BY Price;
+-----+-----+
| Price | COUNT(*) |
+-----+-----+
| 25.00 | 1 |
| 35.00 | 1 |
| 50.00 | 1 |
+-----+-----+
3 rows in set (0.00 sec)
```

Summary

- **SELECT Price, COUNT(*)** → Counts the number of rows for each **Price**
- **GROUP BY Price** → Groups data by **Price**
- **ORDER BY Price** → Sorts the grouped results by **Price**
- Works with **aggregation functions** to summarize grouped data
- **COUNT(*)** → Count all rows in the table or group, including NULLs
- **SELECT *** → Select all columns from a table

SQL HAVING Clause

The **HAVING** clause is used to **specify a search condition for groups or aggregate results**. It is typically used **with the GROUP BY clause** to filter groups after aggregation.

- If **GROUP BY is not used**, HAVING can act like a **WHERE clause** for aggregates.
- HAVING works only with **aggregate functions** like COUNT(), SUM(), AVG(), MAX(), MIN().

Syntax:

```
SELECT column1, column2, AGGREGATE_FUNCTION(column3) FROM table_name
WHERE condition GROUP BY column1, column2 HAVING aggregate_condition
ORDER BY column1, column2;
```

Example:

Suppose we want to **find prices that have more than 1 snack** in the SNACK table:

```
SELECT Price, COUNT(*) AS Snack_Count FROM SNACK GROUP BY Price
HAVING COUNT(*) > 1 ORDER BY Price;
```

```
mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
| 105 | Biscuits | 40 | 35.00 | 2026-11-15 |
| 106 | Nuts | 20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT Price, COUNT(*) AS Snack_Count
-> FROM SNACK
-> GROUP BY Price
-> HAVING COUNT(*) > 0
-> ORDER BY Price;
+-----+-----+
| Price | Snack_Count |
+-----+-----+
| 25.00 | 2 |
| 35.00 | 1 |
| 50.00 | 1 |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

Summary

- **GROUP BY Price** → Groups snacks by their **Price**
- **COUNT(*) AS Snack_Count** → Counts the number of snacks in each price group
- **HAVING COUNT(*) > 1** → Filters groups to show only prices with **more than 1 snack**
- **ORDER BY Price** → Sorts the grouped results by **Price**

SQL ORDER BY Clause

The **ORDER BY** clause is used to **sort the result set** of a query in **ascending (ASC)** or **descending (DESC)** order.

- By default, the **ORDER BY** clause sorts in **ascending order**.
- To sort in **descending order**, the **DESC** keyword is used.
- Multiple columns can be specified to **sort by more than one column**.

Syntax:

```
SELECT column1, column2, ...FROM table_name WHERE condition ORDER BY column1
[ASC|DESC], column2 [ASC|DESC], ...;
```

Example:

1. Sort snacks by name in ascending order (default):

```
SELECT * FROM SNACK ORDER BY Snack_Name;
```

```
mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
| 105 | Biscuits | 40 | 35.00 | 2026-11-15 |
| 106 | Nuts | 20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM SNACK ORDER BY Snack_Name;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 105 | Biscuits | 40 | 35.00 | 2026-11-15 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 106 | Nuts | 20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

2. Sort snacks by price in descending order:

SELECT * FROM SNACK ORDER BY Price DESC;

```
mysql> SELECT * FROM SNACK ORDER BY Price DESC;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
| 105 | Biscuits | 40 | 35.00 | 2026-11-15 |
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 106 | Nuts | 20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

3. Sort snacks by price ascending, then by quantity descending:

SELECT * FROM SNACK ORDER BY Price ASC, Quantity DESC;

```
mysql> SELECT * FROM SNACK ORDER BY Price ASC, Quantity DESC;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 106 | Nuts | 20 | 25.00 | 2026-12-10 |
| 105 | Biscuits | 40 | 35.00 | 2026-11-15 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Summary

- **ORDER BY Snack_Name** → Sorts rows alphabetically by **Snack_Name**
- **ORDER BY Price DESC** → Sorts rows from **highest to lowest Price**
- Multiple columns allow **hierarchical sorting**
- Default sorting is **ASC (ascending)** if no keyword is specified

SQL Constraints

A **constraint** in SQL is a **rule applied to a column or table** to enforce **data integrity** and **consistency**.

Constraints restrict the kind of data that can be inserted, updated, or deleted, ensuring the database remains **accurate and reliable**.

Key Points about Constraints

1. **Enforce rules** on data (e.g., no NULLs, unique values).
2. **Ensure integrity** of relationships between tables (e.g., foreign keys).
3. Can be applied **while creating a table or after the table is created** using ALTER TABLE.
4. Common SQL constraints:
 - **NOT NULL** – Column cannot be empty.
 - **UNIQUE** – Values in the column must be distinct.
 - **PRIMARY KEY** – Uniquely identifies each row (NOT NULL + UNIQUE).
 - **FOREIGN KEY** – Ensures values correspond to primary key in another table.
 - **CHECK** – Ensures values meet a condition (e.g., COST > 0).
 - **DEFAULT** – Provides a default value if none is specified.

1. NOT NULL Constraint

- Ensures that a column **cannot have NULL values**.
- Use it when a value is **mandatory**.

Syntax:

```
CREATE TABLE SNACKS (SNACK_ID INT NOT NULL, SNACK_NAME VARCHAR(50), COST DECIMAL(5,2));
```

Snack Example:

- SNACK_ID is **NOT NULL**, so every snack must have an ID.
- Cannot insert a snack without SNACK_ID.

-- This will fail because SNACK_ID is missing

```
INSERT INTO SNACKS (SNACK_NAME, COST) VALUES ('Chips', 20);
```

```
mysql> SELECT * FROM SNACKS;
+-----+-----+-----+
| SNACK_ID | SNACK_NAME | COST |
+-----+-----+-----+
| 1 | Cookies | 30.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COST) VALUES (1, 'Cookies', 30);
Query OK, 1 row affected (0.07 sec)

mysql> INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COST) VALUES (1, 'Cookies', 30);
Query OK, 1 row affected (0.07 sec)

mysql> SELECT * FROM SNACKS;
+-----+-----+-----+
| SNACK_ID | SNACK_NAME | COST |
+-----+-----+-----+
| 1 | Cookies | 30.00 |
| 1 | Cookies | 30.00 |
| 1 | Cookies | 30.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

2. UNIQUE Constraint

- Ensures that **all values in a column are distinct**.
- Can apply to a **single column** or a **combination of columns**.

Syntax:

```
CREATE TABLE SNACKS (SNACK_ID INT UNIQUE, SNACK_NAME VARCHAR(50), COST DECIMAL(5,2));
```

Snack Example:

- SNACK_ID must be unique; no two snacks can have the same ID.
- Trying to insert a duplicate ID will fail.

-- Will fail if SNACK_ID 1 already exists

```
INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COST) VALUES (1, 'Cookies', 30);
```

```
mysql> CREATE TABLE SNA (SNACK_ID INT UNIQUE, SNACK_NAME VARCHAR(50), COST DECIMAL(5,2));
Query OK, 0 rows affected (0.13 sec)

mysql> INSERT INTO SNA (SNACK_ID, SNACK_NAME, COST) VALUES (1, 'Cookies', 30);
Query OK, 1 row affected (0.08 sec)

mysql> SELECT * FROM SNA;
+-----+-----+-----+
| SNACK_ID | SNACK_NAME | COST |
+-----+-----+-----+
| 1 | Cookies | 30.00 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> INSERT INTO SNA (SNACK_ID, SNACK_NAME, COST) VALUES (1, 'Cookies', 30);
ERROR 1062 (23000): Duplicate entry '1' for key 'sna.SNACK_ID'
mysql> |
```

3. PRIMARY KEY Constraint

- A combination of **NOT NULL + UNIQUE**.
- Uniquely identifies each row in a table.

Syntax:

```
CREATE TABLE SNACKS (SNACK_ID INT PRIMARY KEY, SNACK_NAME  
VARCHAR (50), COST DECIMAL (5,2));
```

Snack Example:

- SNACK_ID is the **primary key**.
- Cannot be NULL and must be unique.
- Ensures every snack has a unique identifier.

-- Allowed

```
INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COST)  
VALUES (1, 'Chips', 20);
```

-- Not allowed: duplicate SNACK_ID

```
INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COST)  
VALUES (1, 'Cookies', 30);
```

4. FOREIGN KEY Constraint

- Ensures **referential integrity** between two tables.
- Values in the foreign key column must **exist in the referenced primary key** column of another table.

Syntax:

```
CREATE TABLE COMPANIES (COMPANY_ID INT PRIMARY KEY,  
COMPANY_NAME VARCHAR(50));
```

```
CREATE TABLE SNACKS (SNACK_ID INT PRIMARY KEY, SNACK_NAME  
VARCHAR(50), COMPANY_ID INT, COST DECIMAL(5,2), FOREIGN KEY  
(COMPANY_ID) REFERENCES COMPANIES(COMPANY_ID));
```

Snack Example:

- SNACKS.COMPANY_ID must exist in COMPANIES.COMPANY_ID.
- Cannot insert a snack with a company that does not exist.

-- Allowed

```
INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COMPANY_ID, COST)  
VALUES (1, 'Chips', 101, 20);
```

-- Not allowed: COMPANY_ID 105 does not exist

```
INSERT INTO SNACKS (SNACK_ID, SNACK_NAME, COMPANY_ID, COST)  
VALUES (2, 'Biscuit', 105, 10);
```

SQL Aggregate Functions



SQL COUNT Function

The **COUNT()** function is used to **count the number of rows in a table**. It can be applied to **both numeric and non-numeric columns**.

- COUNT(*) → Counts **all rows**, including duplicates and NULLs.
- COUNT(column_name) → Counts **only non-NULL values** in the specified column.
- COUNT(DISTINCT column_name) → Counts **unique non-NULL values**.

Syntax:

COUNT(*) COUNT([ALL|DISTINCT] expression)

Examples (Snack Table):

Assume we have a SNACK table with columns Snack_ID, Snack_Name, Price, and Quantity.

1. Count all rows in the SNACK table

```
SELECT COUNT(*) FROM SNACK;
```

```
mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
| 103 | Cookies | 60 | 25.00 | 2026-10-20 |
| 104 | Chocolate | 30 | 50.00 | 2027-01-15 |
| 105 | Biscuits | 40 | 35.00 | 2026-11-15 |
| 106 | Nuts | 20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+
4 rows in set (0.06 sec)

mysql> SELECT COUNT(*) FROM SNACK;
+-----+
| COUNT(*) |
+-----+
| 4 |
+-----+
1 row in set (0.00 sec)
```

2. Count all rows where Price >= 20

SELECT COUNT(*) FROM SNACK WHERE Price >= 30;

```
mysql> SELECT COUNT(*) FROM SNACK WHERE Price >= 30;
+-----+
| COUNT(*) |
+-----+
|         2 |
+-----+
1 row in set (0.00 sec)

mysql> |
```

3. Count distinct snack names

SELECT COUNT(DISTINCT Snack_Name) FROM SNACK;

```
mysql> SELECT COUNT(DISTINCT Snack_Name) FROM SNACK;
+-----+
| COUNT(DISTINCT Snack_Name) |
+-----+
|                             4 |
+-----+
1 row in set (0.07 sec)
```

4. Count number of snacks for each price group

SELECT Price, COUNT(*) FROM SNACK GROUP BY Price;

5. Count number of snacks for each price group having more than 1 snack

SELECT Price, COUNT(*) FROM SNACK GROUP BY Price HAVING COUNT(*) > 1;

```
mysql> SELECT Price, COUNT(*) FROM SNACK GROUP BY Price HAVING COUNT(*) > 1;
+-----+-----+
| Price | COUNT(*) |
+-----+-----+
| 25.00 |         2 |
+-----+-----+
1 row in set (0.00 sec)
```

SUM() Function

- The SUM() function calculates the total of a numeric column.
- Works only on numeric fields like COST, QTY, PRICE, etc.

```
mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
|         103 | Cookies |         60 | 25.00 | 2026-10-20 |
|         104 | Chocolate |         30 | 50.00 | 2027-01-15 |
|         105 | Biscuits |         40 | 35.00 | 2026-11-15 |
|         106 | Nuts |         20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT COUNT(*) FROM SNACK;
+-----+
| COUNT(*) |
+-----+
|         4 |
+-----+
1 row in set (0.00 sec)
```

1. SUM() – Total cost of all snacks

SELECT SUM(PRICE) FROM SNACK;

```
mysql> SELECT SUM(PRICE) FROM SNACK;
+-----+
| SUM(PRICE) |
+-----+
|      135.00 |
+-----+
1 row in set (0.00 sec)
```

2. SUM() with WHERE – Sum of snacks with quantity > 30

SELECT SUM(COST) FROM SNACKS WHERE QTY > 30;

```
mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+-----+
|      103 | Cookies    |      60 | 25.00 | 2026-10-20 |
|      104 | Chocolate  |      30 | 50.00 | 2027-01-15 |
|      105 | Biscuits   |      40 | 35.00 | 2026-11-15 |
|      106 | Nuts       |      20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT SUM(PRICE) FROM SNACK WHERE QUANTITY > 30;
+-----+
| SUM(PRICE) |
+-----+
|      60.00 |
+-----+
1 row in set (0.00 sec)
```

Explanation: Only Chips (5), Chocolate (4), and Biscuit (6) qualify → 20 + 50 + 10 = 80

3. SUM() with GROUP BY – Sum of cost per company (for QTY>3)

SELECT SNACK_BKP_NEW; SUM(PRICE) FROM SNACK WHERE QTY > 25 GROUP BY SNACK_BKP_NEW;

```
mysql> SELECT * FROM SNACK_BKP_NEW;
+-----+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+-----+
|      103 | Cookies    |      70 | 25.00 | 2026-10-20 |
|      105 | Biscuits   |      50 | 35.00 | 2026-11-15 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+-----+
|      103 | Cookies    |      60 | 25.00 | 2026-10-20 |
|      104 | Chocolate  |      30 | 50.00 | 2027-01-15 |
|      105 | Biscuits   |      40 | 35.00 | 2026-11-15 |
|      106 | Nuts       |      20 | 25.00 | 2026-12-10 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT Snack_Name, SUM(Price) AS Total_Price
-> FROM SNACK_BKP_NEW
-> WHERE Quantity > 25
-> GROUP BY Snack_Name;
+-----+-----+
| Snack_Name | Total_Price |
+-----+-----+
| Biscuits   |      35.00 |
| Cookies    |      25.00 |
+-----+-----+
2 rows in set (0.07 sec)
```

4. SUM() with HAVING – Companies with total cost >= 50

```
mysql> SELECT Snack_Name, SUM(Quantity * Price) AS Total_Value
-> FROM SNACK_BKP_NEW
-> GROUP BY Snack_Name
-> HAVING SUM(Quantity * Price) >= 50;
+-----+-----+
| Snack_Name | Total_Value |
+-----+-----+
| Biscuits   | 1750.00    |
| Cookies    | 1750.00    |
+-----+-----+
2 rows in set (0.07 sec)
```

AVG() Function

- The AVG() function calculates the **average value** of a numeric column.
- Ignores NULL values automatically.

Syntax:

AVG([ALL|DISTINCT] expression)

- ALL → averages all values (default).
- DISTINCT → averages only unique values.

Example:

1. AVG() – Average cost of all snacks

```
mysql> SELECT AVG(PRICE) FROM SNACK;
+-----+
| AVG(PRICE) |
+-----+
| 33.750000  |
+-----+
1 row in set (0.00 sec)
```

2. AVG() with WHERE – Average cost for snacks with QTY > 3

```
mysql> SELECT AVG(PRICE) FROM SNACK WHERE QUANTITY > 30;
+-----+
| AVG(PRICE) |
+-----+
| 30.000000  |
+-----+
1 row in set (0.00 sec)
```

3. AVG() with GROUP BY – Average cost per company

```
mysql> CREATE TABLE SNACK_COSTS (
->     Company VARCHAR(50),
->     Cost INT
-> );
Query OK, 0 rows affected (0.12 sec)

mysql> INSERT INTO SNACK_COSTS (Company, Cost) VALUES
-> ('Lays', 20),
-> ('Lays', 20),
-> ('Oreo', 30),
-> ('Cadbury', 50),
-> ('Parle', 10);
Query OK, 5 rows affected (0.08 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

SELECT COMPANY, AVG(COST) FROM SNACKS_COSTS GROUP BY COMPANY;

```
mysql> SELECT COMPANY, AVG(COST) FROM SNACK_COSTS GROUP BY COMPANY;
+-----+-----+
| COMPANY | AVG(COST) |
+-----+-----+
| Lays    | 20.0000   |
| Oreo    | 30.0000   |
| Cadbury | 50.0000   |
| Parle   | 10.0000   |
+-----+-----+
```

MAX() Function

- Finds the **maximum (largest) value** in a numeric column.

Syntax:

MAX([ALL|DISTINCT] expression)

Example – Maximum cost of snacks:

```
mysql> SELECT MAX(PRICE) FROM SNACK;
+-----+
| MAX(PRICE) |
+-----+
|      50.00 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
|      103 | Cookies    |      60 | 25.00 | 2026-10-20   |
|      104 | Chocolate  |      30 | 50.00 | 2027-01-15   |
|      105 | Biscuits   |      40 | 35.00 | 2026-11-15   |
|      106 | Nuts       |      20 | 25.00 | 2026-12-10   |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

With WHERE (e.g., QTY > 3):

```
mysql> SELECT * FROM SNACK;
+-----+-----+-----+-----+-----+
| Snack_ID | Snack_Name | Quantity | Price | Expiry_Date |
+-----+-----+-----+-----+-----+
|      103 | Cookies    |      60 | 25.00 | 2026-10-20   |
|      104 | Chocolate  |      30 | 50.00 | 2027-01-15   |
|      105 | Biscuits   |      40 | 35.00 | 2026-11-15   |
|      106 | Nuts       |      20 | 25.00 | 2026-12-10   |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT MAX(PRICE) FROM SNACK WHERE QUANTITY > 20;
+-----+
| MAX(PRICE) |
+-----+
|      50.00 |
+-----+
1 row in set (0.00 sec)
```

MIN() Function

- Finds the **minimum (smallest) value** in a numeric column.

Syntax:

MIN([ALL|DISTINCT] expression)

Example – Minimum cost of snacks:

SELECT MIN(COST) FROM SNACKS;

```
mysql> SELECT MIN(PRICE) FROM SNACK;  
+-----+  
| MIN(PRICE) |  
+-----+  
|      25.00 |  
+-----+  
1 row in set (0.00 sec)
```

With WHERE (e.g., QTY > 3):

```
mysql> SELECT MIN(PRICE) FROM SNACK WHERE QUANTITY > 20;  
+-----+  
| MIN(PRICE) |  
+-----+  
|      25.00 |  
+-----+  
1 row in set (0.00 sec)
```